Embedded IoT

Connecting Embedded s/m to the Internet.

* Embedded s/m
* Sensors & Actuators
* microcontroller
* Commu protocols
* cloud & IoT Platforms
* Real-Time Applications

IoT (Internet of things).

It is a technology where Physical devices are Connected to a n/w so they can:

- ↳ collect data
- ↳ Process "
- ↳ Commu "
- ↳ Receive Commands
- ↳ Perform actions automatically

Things → Internet → Data → Decision → Action

Real-life Ex:

* Smart Home * Smart Agri * Smart factory
* Connected vehicles * Healthcare monitoring
* Smart energy meter * Smart city * Wearable Device

Embedded S/m

It is a computer s/m designed to perform a specific function inside a device.

Ex: washing machine, microwave, car ECU, medical equipment, Printer, Digital camera, motor controller

I/P → Processing → O/P

Connecting Embedded S/m to Internet

1) Embedded s/m → purpose-built systems, Real-time operation; Efficient & reliable

2) Sensors & Actuators → sense real world data, Act on the environment, Bridge physical to digital

3) MCU → Process & control, Low power & Compact, core of embedded IoT

4) Commu Protocol → wi-fi, Bluetooth, MQTT, HTTP/HTTPS CoAP
Reliable data exchange, secure commu

5) Cloud & IoT platforms → store, visualize & analyse data
Remote monitoring & control,
scalable & secure.

Embedded Vs IoT

understanding the difference b/w a traditional embedded s/m & an IoT s/m

Feature	Embedded s/m	IoT
1) function	Performs a specific function	Performs a function + commu
2) operation	usually standalone	Network connected
3) Processing	Local processing	Local + cloud processing
4) Data flow	sensor → MCU → Actuator	sensor → MCU → N/w → clo
5) Ex	Digital thermometer	wi-Fi temperature monit

Ex:

Temperature sensor → STM32 (MCU) → wifi module → cloud → mobile App Dashboard

Basic IoT Architecture

It works in layer from the physical world to the application

1) Device layer → sensors, Actuators, MCU, Data acquisition, Local control

2) connectivity layer → wired comm, wireless comm, short range, long range, Gateways

3) N/w / Internet layer → Internet / IP, Routing, Data transfer, Security (SSL/TLS), Reliable comm

4) Cloud / Data Processing layer → Data storage, Data processing, Analytics, Machine learning, Data management

5) Application layer → web dashboards, Mobile apps, Alerts & notifications reports, user interface

IoT data Flow

Sense → Process → Connect → Transmit → Store → Analyze → Act

MCU

It is the main brain of an embedded IoT devices. It reads I/P from sensors, processes the data using its program, & control O/P such as actuators or commu with other devices.

Actuators

It is a device that converts electrical signals from a MCU into physical action.

Communication

this technologies connect IoT devices to each other gateways, & the cloud to exchange data reliable & securely.

1) wifi (IEEE 802.11) → wireless (LAN) → Range - 50m → high data rate (11 MBPS - 1GBPS)
→ High Power Consumption → high data transfer. Video Streaming, firmware updates

2) Bluetooth (BLE) → wireless (PAN) → Range - 10m → Low data rate → (125 KBPS - 2MBPS)
Low Power consumption →

→ ~~smart home, building~~ wearable, health devices, short comm

5) Zigbee (IEEE 802.15.4) → (mesh) → 10-100m → low → low → smart b building automati sensor

4) LoRaWAN → (WAN) → 2-15Km → Low → Very Low → smart Agri, asset tracking smart citie

5) Cellular (2G, 3G, 4G 5G) → WAN → wide area (Km's) → High → Medium - High → Remot monito Mobil real-tim applicat

---

Communication Protocol

that are rules that defines how data is formatted, transmitted, received & understood b/w IoT devices & s

* MQTT * CoAP * HTTP/HTTPS * WebSocket
* AMQP * DDS * LWM2M

Cloud & IoT platforms

It provides the infrastructure & service to, securely connect, manage, store, analyse & act on data from IoT devices

* AWS IoT Core * Microsoft Azure IoT Hub * Google Cloud Io Core
* IBM Watson IoT Platform * Cisco IoT Control Center
* Things Board (Open source)

ESP8266 Wifi module

* It is a low-cost Wi-Fi MCU.
* Developed by Espressif s/m
* used to connect devices to the Internet (IoT)
* Has built-in TCP/IP Protocol stack

Key features

* MCU : Tensilica 32-bit RISC CPU xtensa LX106
* operating vtg : 3.3V
* I/P vtg : 7-12V (via VIN Pin) or 5V (via micro-USB)
* clock speed : 80MHz (adjustable up to 160 MHz)
* Flash memory : 4MB
* SRAM : 64 KB (instruction), 96 KB (data)

Wifi - capabilities

* standard : 802.11 : b/g/n
* Freq : 2.4GHz
* modes : station (connects to router), SoftAP (Act as a router) or Both simultaneously
* Security : supports WPA/WPA2 encryption
* N/w Protocols : IPV4, TCP/UDP/HTTP/HTTPS/FTP

Overviews of Peripheral

* GPIO : 16 General Purpose I/P / O/P Pins
* PWM. Pulse Width modulation available on all digital Pins (S/w - based)
* ADC : 1 Analog to Digital Converter Pin (10-bit resolution)
* Comm : UART, SPI, & I2C interfaces

PIN Diagram Explanation

* Vcc → Power supply (3.3 V only)
* GND → Ground
* TX/RX → Serial Comm.
* GPIO 0 – GPIO 16 → O/P / I/P Pins
* EN (CH_PD) → Enable Pin.
* RST → Reset module

working Principle

- ESP8266 connects to a wi-fi n/w
- uses TCP/IP Protocol to send / receive data
- Can acts as :
  - ↳ client (send data to server)
  - ↳ server (host web page)

Program

```
#define LED D4
void setup (){
  Pin Mode (LED, OUTPUT);
}
```

```
void loop() {
  digitalWrite(LED, LOW); // Turn LED ON
  delay(1000);
  digitalWrite(LED, HIGH); // Turn LED OFF
  delay(1000);
}
```

Advantages :-

* Low cost
* Built-in Wi-Fi (No external module needed)
* Easy to program using Arduino IDE
* Small size & fast processing
* Huge community support.

Disadvantages

* Limited GPIO Pins
* Needs stable 3.3V supply
* Not suitable for heavy processing tasks
* Wi-Fi consumes more power

Compilers / IDEs

* Arduino IDE
* Platform IO
* ESP-IDF

Programming Languages:

* C
* C++

Application

* Smart home automation
* IoT - based monitoring s/m
* weather stations
* Wireless sensor n/w
* Remote control s/m

ESP8266 : Deep Dive into Timers, Interrupts & ADC

The ESP8266 MCU : ~~#Brief~~

Timers : Precision control & multitasking

Interrupts : Events - Driven programming

ADC : Bridging physical & Digital worlds

Practical Application : How the code works

Beyond just wi-fi

* usually called as cheap wi-fi module, but it is a powerful 32-bit MCU (Tensilica L106)
* operates at 80 MHz (overclockable to 160 MHz)
* To build responsive, professional-grade IoT devices, we must move past basic delay() function
* the key: utilizing Timers, Interrupts & the ADC.

what is a Timer

Allows the ESP8266 to perform tasks at exact, regular intervals

they eliminate the need for delay(), which block the CPU & crashes wi-fi stacks

Timer architecture

H/W Vs S/W Timers

* Timer 0: used by the ESP8266's internal wi-fi stack. Do not touch this

* Timer 1: Available for user application, highly precise H/W timer

* S/W Timers (OS Timers): Provided by the ESP8266 RTOS / Arduino core (E.g., the Ticker library). Great for tasks that don't need microsecond precision

The Ticker library (S/W Timers)

* A built-in library for the ESP8266

* Automatically calls a specific function at a set interval

* Runs asynchronously - your loop() function continues running freely

* Best for: Sending MQTT data every 10 seconds, blinking status LEDs, checking sensor health.

Program Logic - The Ticker Blinker

what this Program does:

* Objective: Blink a status LED every 500ms without blocking the main loop.

* Mechanism: The code creates a Ticker object. It tell the Ticker to trigger a "toggle LED" function every 0.5 sec.

* Results: the main loop() is completely empty. The LED blinks perfectly in the background while the ESP8266 is free to handle wi-fi & other tasks.

Polling Vs Interrupts

* Polling: if (digitalRead(button) == HIGH) inside loop()

↳ wastes CPU cycles ↳ might miss a fast button press if the CPU is busy doing something else

* Interrupts: Catches 100% of events instantaneously extremely power-efficient

ESP8266 GPIO Interrupt

* All pins in ESP8266 support H/w interrupts
* Configures using attachInterrupt(pin, ISR, mode)
* Connect to buttons, motion sensors, flow meters,

rotary encoders.

Modes:

Raising : LOW → HIGH
Falling : HIGH → LOW
Change : both rising & falling (change any state)

ADC :

ESP32 → 12 bit ADC
ESP8266 → 10 bit ADC

* Digital Range : O/P values 0 to 1023
* Native voltage Range : The bare ESP8266 chip only supports 0V to 0.1V max on its ADC Pin
* Warning : Exceeding 1.0V on bare ESP8266 chip will damage it

The A0 Pin & Dev Boards

NodeMCU/Lemos D1 mini advantages

* Development boards (like NodeMCU) have a built in Vtg divider circuit
* This scales down the i/p Vtg, safely allowing 0V to 3.3V, to be applied to the A0 Pin
* Reading the Pin : Achieved simply by calling analogRead(A0)

AWS

Reading the Data

The conversion math

* If connected to 3.3 V (on a NodeMCU)

0 volts = 0 (Digital Value)

1.65 " ≈ 512 (" ")

3.3 " ≈ 1023 (" ")

* Formula to get Vtg : Voltage = (ADC_Value / 1023.0) * 3.3

Program Logic - Analog LED Dimmer

what this Program does:

* objective : Control the brightness of an LED using a physical potentiometer knob

* mechanism : the Loop() continuously reads the A0 pi It maps the incoming 0-1023 ADC value to a 0-255 PWM) value

* Result : As the user turns the knob, the analog Vtg changes, the ADC digitizes it, & the LED brightness smoothly scales up or down in real-time

Program Logic - Internal Vcc Monitor

* objective : measure the battery level powering the ESP8266 without any external wiring to A0

* mechanism : By setting a specific macro (ADC_MODE (ADC_VCC). the ESP8266 disconnects A0 from the outside world & routes it to be internal power supply

Reading the Data

The conversion math

* If connected to 3.3 V (on a NodeMCU)

0 volts = 0 (Digital Value)

1.65 " ≈ 512 (" ")

3.3 " ≈ 1023 (" ")

* Formula to get Vtg : Voltage = (ADC_Value / 1023.0) * 3.

Program Logic - Analog LED Dimmer

what this Program does:

* objective : control the brightness of an LED using a physical potentiometer knob

* mechanism : the loop() continuously reads the A0 p It maps the incoming 0-1023 ADC value to a 0-255 PWM) value

* Result : As the user turns the knob, the analog Vtg changes, the ADC digitizes it, & the LED brightnes Smoothly scales up or down in real-time.

Program Logic - Internal Vcc Monitor

* objective : Measure the battery level powering th ESP8266 without any external wiring to A0

* mechanism : By setting a specific macro (ADC_MODE (ADC_VCC), the ESP8266 disconnects A0 from the outside world & routes it to be internal power supply

real, the Program

* Result: A battery-powered IoT device can email you a "Low Battery" warning right before it dies, utilizing internal ADC routing

Putting It All Together

The Ultimate IoT Architecture

* Interrupt: Handle immediate user inputs & fast sensors
* ADC: Periodically gathers environments data
* Timer: Orchestrates everything - sends the ADC data over Wi-Fi every 5 minutes & flashes a status LED every sec
* Result: A rock-solid, crash-free ESP8266 application

Universal Asynchronous Receiver - Tx

* One of the oldest & simplest serial communication protocols
* Full-duplex - send & receive simultaneously

Hw Serial Ports

* UART0: Pins Tx (GPIO 1) & Rx (GPIO 3). used by the USB connection for flashing & serial monitor
* UART1: Pin Tx (GPIO 2). Rx is occupied by internal flash memory. Transmit only
* S/w serial: A library that allows any digital pins to act as UART, freeing up to USB Hw serial for debugging

I2C

* A synchronous, multi-master, multi slaves packet switched, singled ended serial bus
* Two line - SDA, SCL
* Ideal for connecting multiple sensors to a MCU over short distances

S/w Implementation

* The ESP826L does not have h/w I2C peripheral
* It uses "bit-banging" to emulate the protocol
* Pins : D1 (SCL) & D2 (SDA)
* Addressing : Each slave device on the bus has a unique 7-bit hexadecimal address

Bus Topology

* Connection : SDA to SDA, SCL to SCL
* Common Ground : GND to GND
* Pull-up resistors : I2C requires standard 4.7kΩ pull-up resistor on both SDA & SCL lines connected to 3.3V (often it handle by ESP's internal pull-ups for short distances - but external is safer)

Serial peripheral Interface

* A synchronous serial communication interface specification used for short-distance communication primarily in embedded S/m
* High speed : much faster than I2C & UART

* Four wires : MOSI
    * MISO
    * SCLK
    * MOSI
    * CS/SS

Hw SPI

* ESP8266 has two SPI blocks : SPI & HSPI
* the primary SPI is dedicated to the external flash memory
* HSPI : Available to the user :
    * SCK : D5
    * MISO : D6
    * MOSI : D7
    * CS : D8

master / slave architecture

* Connect identical pins : MOSI to MISO, MISO to MOSI SCK to SCK
* Connect the master's CS pin to the slave's CS Pin
* Connect GND to GND
* ESP8266 SPI slave mode required a specific library (SPISlave) as it handles data via H/W registers differently than an arduino uno.

Wireless Protocol stack for IoT

1) Physical layer → Physical medium (Radio freq)
Transmission of raw bit over the medium — Ex: 2.4 GHz, 868/915 MHz, Sub-GHz Band, optical, etc.

2) Data link / Adaptation Layer (node-to-node delivery & MAC addressing) → IEEE 802.15.4 | 6LoWPAN | BLE | Wi-fi (802.11) | RFID/NFC | 3G/4G/5G

3) N/w layer
Logical addressing & routing → IPV4, IPV6, RPL

4) Transport layer → TCP, UDP

5) Application Layer → HTTP/HTTPS, MQTT, CoAP

* wireless Protocols are sets of rules & standards that allows devices to communi with each other either Physical cables using radio waves or other wireless sig

1) Wi-fi - IEEE 802.11 → high-speed wireless N/w for Internet & LAN Connectivity

Key Technical points :-

* Standard : IEEE 802.11 family
* Freq Bands : 2.4 GHz, 5 GHz, & 6 GHz
* Data Rate : up to multiple GBPS
* Topology : star
* Medium Access : CSMA/CA (Carrier sense multiple Acc with collision Avoidance)

* Security : WPA 3 (latest), WPA 2, WPA (legacy)
* Modulation : DSSS, OFDM, MU-MIMO, OFDMA (Wi-Fi 6/6E)
* Typical Range : 30m - 100m indoors

2) Bluetooth → Short-range wireless communication Technology used to connect nearby devices without cables. It works at 2.4 GHz & is used in headphones, speakers, keyboards, mice, phones, & smartwatches. Its range is usually around 10 metres but some devices can work up to 100 metres

3) Bluetooth Low Energy (BLE)

BLE is a low-power version of Bluetooth. It is designed for devices that need long battery life, such as fitness bands, smartwatches, health sensors, & IoT devices. It sends small amounts of data while using very little power

4) Z-wave

Z-wave is a wireless protocol mainly used in smart homes. It connects smart lights, locks, sensors, thermostats & security devices. It uses a mesh n/w, where devices help each other send signals over a wider area

5) LoRa & LoRa WAN

LoRa is a long-range & low-power wireless technology for IoT devices. LoRa WAN is the n/w protocol

used with LoRa devices. It is used in smart agriculture, smart cities, water meters, asset tracking & industrial monitoring

5) RFID

It stands for radio frequency Identification. It uses radio waves to identify & track objects using RFID tags & readers. It does not need direct line of sight like barcodes. It is used in shops, attendance sys, access cards, hospitals & logistics

6) Infrared (IR)

It communication uses invisible light to send data over short distances. The transmitter & receiver must face each other because IR needs a clear line of sight. It is used in TV remotes, AC Remotes, DVD players & home appliances

7) cellular 4G & 5G

4G & 5G are mobile n/w technologies that provide Internet & comm services. 4G gives fast internet, while 5G gives higher speed, lower delay, & supports more connected devices. They are used in mobile phones, smart cities, connected vechicles & IoT

8) NFC

It stands for Near Field Comm. It allows two devices to exchange data when they are very close, usually within 4cm. It is used in contactless payments, access cards, ticketing, device pairing & data sharing

9) TCP & UDP

TCP & UDP are Protocols used to send data through a n/w. TCP is reliable because it checks whether all data is delivered correctly. UDP is faster but does not guarantee delivery. TCP is used for websites & emails. while UDP is used for video streaming, gaming & voice calls.

10) HTTP & HTTPS

HTTP is used for commu b/w a web browser & a web server. HTTPS is the secure version of HTTP. HTTPS encrypts the data & protects passwords, Payments details & personal info. HTTPS is safer for website

11) FTP

It stands for File Transfer Protocol. It is used to upload & download files b/w a Computer & a server. FTP uses separate connections for Commands & file transfer. Secure versions such as SFTP & FTPS are better for protected file sharing

12) IPV4 & IPV6

IPV4 & IPV6 are used to give addresses to devices on a n/w. IPV4 uses a 32-bit address, such as 192.168.1.1

IPV6 uses a 128-bit address & Provides many more IP addresses. IPV6 is useful for future internet growth & IoT devices.

13) DNS, DHCP & other protocols

DNS converts website names into IP addresses. DHCP automatically gives IP addresses & n/w settings to devi other protocols like NTP, ICMP, & ARP help with time synchronization, n/w testing & device commu.

ThingSpeak

* ThingSpeak is an IoT analytics platform used to collect, store, analyse & visualise IoT data in the clo

working :-

Sensor → IoT Device → Internet → ThingSpeak Cloud → Analysis Visualization

Key concepts

1) channel :-

stores IoT data. It supports up to 8 data fields. 3 location fields & 1 status fields

2) Field: stores individual data such as temperature, humi light, etc.

3) write API key: used to send data from a device/appli to a ThingSpeak channel.

4) Read API Key: used to ~~send~~ read data from a ThingSpeak Channel

5) Data visualization: shows data using charts, graphs & dashboards.

6) APPs & integration: supports alerts, MATLAB analysis & integration with other services Benefits

1)

Benefits:

Easy to use and free for basic usage.
No server or data base management
Real-time data visualization.
Powerful analytics with MATLAB.
Scalable for IoT application
Sensor -> cloud -> Insights
Turn Data into a Smarter Tomorrow.

Key Features:

Data collection & storage in the cloud.
Real-time charts & dashboards.
MATLAB analytics integration.
Alerts via email, Twitter, Telegram, Twilio, etc.
RESTful API for easy integration.
CONNECT -> MONITOR -> INNOVATE

Supported protocols;

HTTP | HTTPS -> web-based communication
MQTT (via MQTT API). Lightweight IoT messaging.
ThingSpeak API (REST) send and retrive data from ThingSpeak.
Arduino, ESP8266, Raspberry pi, NodeMCU, STM32, etc.

AWS IoT : Connection Embedded Devices to the cloud.

Aws IOT enables physical devices to communicate with cloud Service.

Used in IoT, embedded systems, smart homes, industrial automation, healthcare, agriculture, automotive, etc.

Embedded Device -> internet -> AWS cloud -> Application.

Embedded -> Internet -> AWS cloud -> Application

Important AWS IoT Components:

These components work together to connect Secure, Communication and Manage your IOT devices in AWS.

Thing -> Represents a real IoT devices in AWS, like a sensor (or) Machine.

Thing Register - Stores information about all the IoT devices

Devices Certificate - Proves that the device is trusted and allowed to connect to AWS.

Policy - Defines what actions the Device is allowed or not allowed to do.

MQTT - A lightweight communication protocol used for sending Message b/w IoT devices and AWS.

Topic -> A named channel where devices publish and subscribe to message.

Device Shadow -> A cloud copy of the device's current state. It helps AWS know the devices status even when the device is offline.

Rules Engine. Takes messages from devices and sends them to other AWS services for processing.

IoT Endpoint - The AWS address (URL) that an IoT device uses to connect securely to AWS IoT Core.

What is CoAP:

CoAP stands for Constrained Application Protocol.

- It is a lightweight protocol used in IoT
  It is designed for devices with low memory, low power and limited processing.
  It follows a client - server model.
  It is similar to HTTP, but it is lighter and faster.

CoAP normally uses UDP.

Ex: Client - "Give me the temp".
Server - Temp is 30°C.
So, the main idea is:
Client request -> Server responds.

CoAP

Constrained Application protocal

* Constrained process power (like)
* Constrained Memory (like).
* Constrained energy source (low)

Kid of device.
IOT / M2M devices

there are so many different applications protocol & out there what kind of application protocol you mean here.
Similar to HTTP
But much simpler
(based on REST Mo

MQTT Architecture:

MQTT follows a publish / subscribe model.

3 main parts.

Publisher:
Sends data to a topic.
Ex: ESP32 temp sensor.
It publishes: factory/temp = 30°C.

MQTT Broker:
Receives the data from the publisher.
It checks the topic
It sends the data to the correct subscribe

Subscriber.
Subscribes to a topic
Receives the data from the broker.
Ex: Node-RED, Cloud platform, mobile app.

Simple flow:
Sensor → publish → Broker → Subscribe → app

MQTT publish & subscribe:
An ESP32 has a temperature sensor.

If publishes:
Topic: factory/temp, Message: 30°C.
then: MQTT Broker receives it.
then: Node-RED receives it, Cloud platform receives
Mobile app receives it.
publisher and subscribers do not need to know each
other, The Broker manages the communication
MQTT QoS: => Quality of service, Send
It has 3 levels: - QoS 0 - At most once,
Message is sent only once, No confirmation,
It may be lost, fastest
QoS 1 -> At least once => Receiver sends Ack,
Msg is delivered at least once. Duplicate Msgs can
happen.
QoS - 2 => Exactly once, Highest reliability
Message is delivered exactly once, No duplicate
Me, More Comm is required.
CoAP Msg types: 4 Message type:
CON -> Confirmable => requires acknowledgement.
used when reliable delivery is important.
NON - Non-confirmable => Does not acknolgm
used when reliability is not very important.
ACK - Acknowledgement :=> Confirms that a CON Msg
was received.
RST - Reset:
Indicates that the Msg could not be
processed.

Raspberry Pi Programming

* single board computer developed by the Raspberry Pi foundation (UK), Originally built to teach programming in schools, now widely used for hobbyist & professional H/W projects. (~~small~~ Credit-card sized Computer)

Real-life Ex: smart-home, CCTV (surveillance), Robot, weather station, Home server.

History of Raspberry Pi

Pi 1 → 2012, the original	single-core ARM processor 256 MB RAM Proved a full Linux PC could cost just $25	
Pi 2 → 2015, Performance Boost	Quad-core ARM Cortex-A7 1GB RAM Major speed boost for Real Projects	
Pi 3 → 2016, Connectivity upgrade	Built-in wifi " Bluetooth Huge upgrade for IOT & wireless Projects	
Pi 4 → 2019. Power, versatility	USB 3.0 Dual 4K HDMI support up to 8GB RAM usable as a desktop replacement	
Pi 5 → 2023, Most-Gen Performance	Faster CPU & GPU PCIe support Improved power efficiency current flagship - near top-level performance	

PC	Feature	Raspberry Pi
1) Large	Size	small
2) Expensive	Cost	Affordable
3) Higher performance	Performance	Lower performance
4) x86 CPU (Intel/AMD)	CPU architecture	ARM CPU
5) Higher electricity use	Power consumption	Lower electricity use
6) Runs Windows/Linux/macOS	OS	Runs Linux (Raspberry Pi OS, Ubuntu, etc)
7) Standard SATA/NVMe storage (HDD/SSD)	Storage	SD card (or USB)
8) Expandable (RAM, GPU, storage, add-on cards)	Upgradability	Mostly fixed h/w, limited upgrade
9) General-purpose computing (work, gaming, design etc)	Best for	Learning, IoT, automation, hobby projects, prototyping

Node - RED for IOT

Visual programing for IOT, Automation & Dashboards

Sensors → MQTT → Node-RED → Processing → Dashboard
↓
Actuators

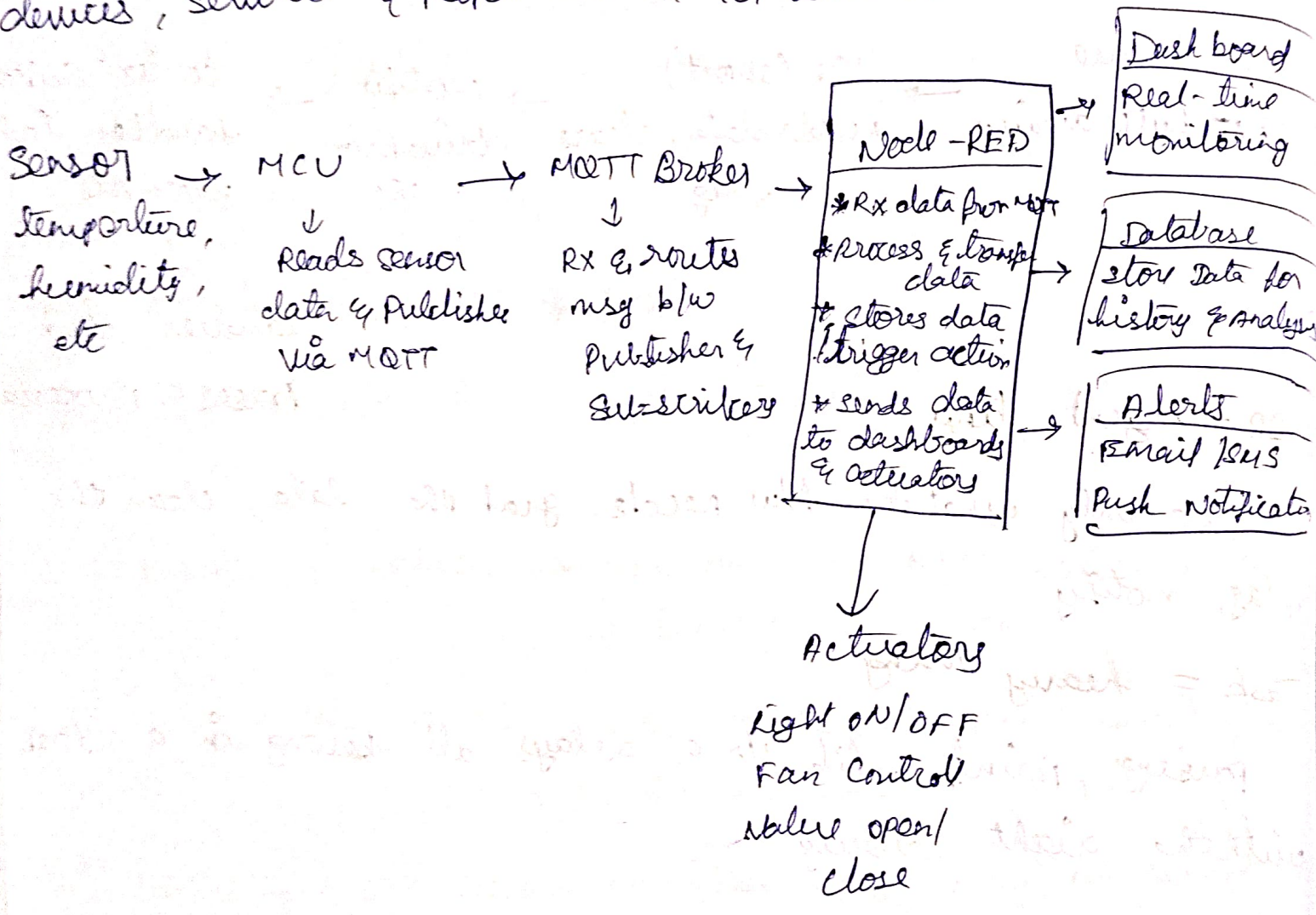
ode-RED

It is flow based - programming tool used to connect
w devices, APIs, services & online platforms

Instead of writing everything as traditional code,
pplication can be created by connecting nodes together in
Visual flow.

connect → configure → Deploy.

~~the~~ Node-RED act as the central hub that connects devices, services & people in an IoT solution



where does Node-RED Runs

It is highly flexible & can run on many platfor
& environments

* window PC * Linux PC * Raspberry Pi
* Cloud server * Docker * Industrial Computer

Architecture

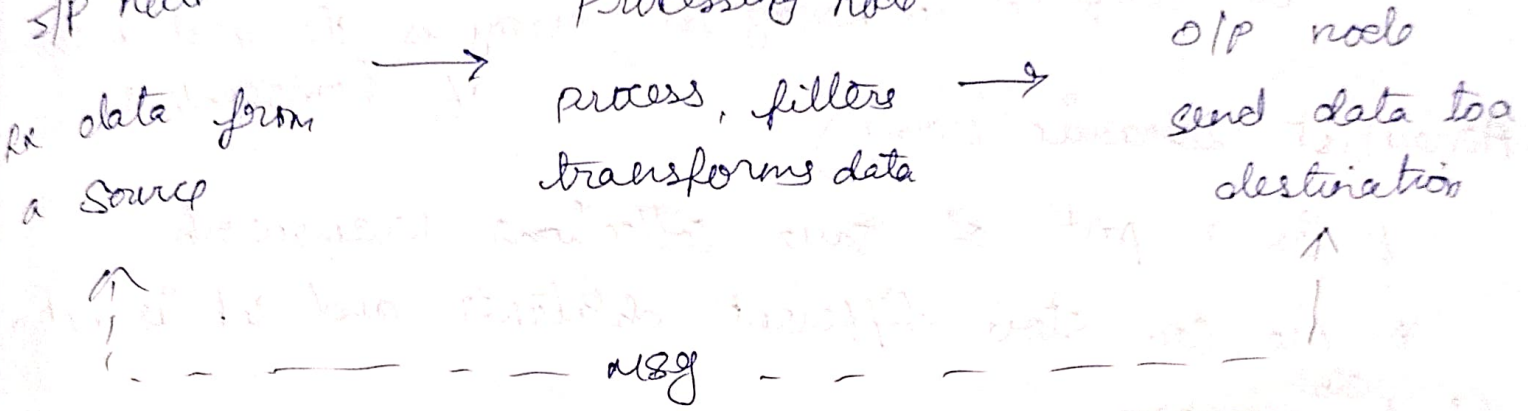
Node-RED applications are built using three major
concepts that work together

1) Nodes → perform individual operation
Each node has a specific functio

2) Flows → connect nodes together to creat
a workflow that performs a task

3) Msg → Carry info b/w nodes, msg flow
from one node to another

How It works



Info flows as a msg